

For this, let us run two instances of UML in two terminals. We need to identify each UML instance and use this identification while configuring it. This is done using 'umid' as a parameter while starting the UML instance. For our example of UML instances, let's use 'UML1' and 'UML2' as identities. Run the following commands to start these instances.

For the first instance, named 'UML1', use the following command:

```
./linux ubd0=cow1,debian_root_fs umid=UML1
```

For the second instance, named 'UML2', use the command shown below:

```
./linux ubd0=cow2,debian_root_fs umid='UML2'
```

After running these instances, type *ifconfig -a* in each instance. You will see that none of these instances have any network interface except loopback. There are many ways in which you can add network interfaces to these instances. We will look at two methods – the multi-cast and the TUN/TAP methods. If you need to have communication between multiple instances of UML, but don't need these UML instances to communicate with the external world, then choose the multi-cast method. In this method, each interface of UML will subscribe to a local multi-cast group, so any packet sent out of a UML instance will reach every other instance of the UML that is subscribed to this group. The disadvantage of this is that it works like a hub, wherein if there are three instances of UML, each is subscribed to a multi-cast group and then any packet sent to that group will reach all these instances. There cannot be a switch or bridge-like configuration, whereby packets just reach a particular node. For such a configuration, you will need to use the TUN/TAP method.

Multi-cast interfaces

To add a multi-cast Ethernet interface on a UML instance, use the *uml_mconsole* utility. This is the utility to configure each UML instance, with which each instance is identified using its *umid*. From the host terminal (this is the terminal other than the one from which UML is running), to add an Ethernet interface to each UML interface, type the following command replacing the *umid* of the respective UML:

```
uml_mconsole <umid> config eth0=mcast
```



Note: You may need to have root permissions to execute the above command.

Now in each UML instance, type the *ifconfig -a* command and you will notice that the *eth0* interface is added to each instance but is not up and running. To start

these interfaces, assign an IP address to the *eth0* interface of each UML instance. Let's use 192.168.1.1 for UML1 and 192.168.1.2 for UML2. Type the following commands in the UML instance terminal.

In the UML1 instance:

```
ifconfig eth0 192.168.1.1 up
```

In the UML2 instance:

```
ifconfig eth0 192.168.1.2 up
```

Now, using the *ifconfig* command, you can see that the interfaces in each UML instance are up and running. You will also notice how easy it is to add an Ethernet interface in a virtual UML platform. Now, since *eth0* interfaces are connected to the same multi-cast group, they can communicate with each other. We can test this by pinging each. First, start *tcpdump* from UML1 and ping UML1 from UML2. You will notice that the pinging will start getting responses, and you can observe that in *tcpdump* UML1 is getting packets from UML2, and is also sending responses to UML2.

From UML1, issue the following command:

```
tcpdump -ni eth0
```

From UML2, use the following command:

```
ping 192.168.1.1
```

You can also test this the other way around. The network we just created is an isolated one and will not affect any communication in your host network. Now you can write any network program and communicate between these instances. You can even start another UML instance, give it an Ethernet interface, configure its IP address and test anything on it as you would do on your PC. Congratulations! You are now the network administrator of this new isolated network.

Now, let's look at how to add a new Ethernet interface *eth1* on each instance with a different multi-cast group, so that communication on *eth0* will not interfere with *eth1*. On the host terminal, type the following command for each UML instance:

```
uml_mconsole <umid> config eth1=mcast,,239.192.168.1,1103,1
```

After *mcast*, the first parameter is the MAC address. Leave it empty so that the system configures it to some random value. The third parameter is the multi-cast group address to join, and the fourth parameter is the port on which these interfaces will communicate (1102 is the default port, which was assigned to *eth0*; we have chosen 1103 for *eth1* so that these don't interfere, since we have chosen the same multi-cast group for both *eth0* and *eth1*).